SISTEMA FCT · GUÍA PARA APRENDER

Más orden, menos consultas

Una explicación fácil de la paginación y de los cambios en ofertas, matching y asignaciones.

La idea en una frase

En lugar de cargar listas enteras y consultar la base repetidamente por cada oferta, se muestran porciones pequeñas y se agrupan consultas relacionadas.

Qué encontrarás aquí

- Qué problema había y por qué importaba.
- Qué hace cada cambio, explicado paso a paso.
- Dónde está cada pieza en el proyecto.
- Cómo probarlo y qué falta verificar con MariaDB.

Preparado para Practicas_Desarrollo · 15 de octubre de 2026

1. El problema, sin tecnicismos

Imagina una lista con 30 ofertas. La página necesita mostrar cada oferta y cuántas plazas quedan.

Antes, el programa hacía esto:

```
1 consulta para traer ofertas
+ 1 consulta de plazas por cada oferta
+ 1 consulta de candidatos por cada oferta con plazas
```

Ese patrón se llama **N+1**: primero se consulta una lista y luego se repite otra consulta por cada elemento. Si crecen las ofertas, crece también el número de viajes a la base de datos.

¿Qué significa paginar?

Dividir una lista larga en páginas. Por ejemplo, si hay 73 asignaciones y se muestran 20 por página, el usuario navega entre cuatro páginas. La base devuelve solo las filas de la página actual.

¿Qué cambia?

Antes	Ahora
Se cargaba toda la lista en memoria.	Se carga una página de resultados.
Matching repetía consultas oferta por oferta.	Los cupos se agrupan y los candidatos se buscan para varias ofertas juntas.
El panel cargaba registros para luego contar algunos.	La base calcula totales y medias; el panel pide solo lo que muestra.
Se podía pensar en añadir índices antes de medir.	Primero se mide; un índice nuevo requiere evidencia.

> **Importante:** paginar reduce las filas que se envían a la página. En matching la consulta sigue comprobando las reglas reales de compatibilidad (ciclo, fechas, turno, horario y asignaciones activas); no sustituye esas reglas.

2. Cómo recorre el sistema una página

1. **La URL indica la página.** Por ejemplo, `?pagina=2`. El sistema valida el número para que valores inválidos vuelvan a la primera página.
2. **El modelo cuenta cuántos elementos hay.** Así puede mostrar "Página 2 de 5".
3. **El modelo carga solo esa parte.** SQL usa `LIMIT` y `OFFSET` para seleccionar el tamaño de página y desde qué fila empezar.
4. **La vista muestra los resultados y los enlaces.** "Anterior" y "Siguiente" conservan la acción y, en matching, las páginas independientes de cada oferta.

Matching en una página

```
A. Cargar hasta 10 ofertas aprobadas vigentes
B. Consultar en grupo cuántas plazas están ocupadas
C. Buscar candidatos compatibles de esas ofertas en una consulta
D. Mostrar hasta 10 candidatos por oferta y permitir navegar
```

Para separar la lista de candidatos de cada oferta, SQL asigna un número de fila con `ROW_NUMBER()`. La navegación por candidatos no cambia la página de ofertas ni la de las otras tarjetas.

La operación de asignar sigue siendo una acción distinta: vuelve a validar la oferta, el cupo y el candidato dentro de una transacción antes de guardar. Ver candidatos no reserva plazas.

¿Por qué no agregar índices ahora?

Un índice es como el índice de un libro: puede ayudar a encontrar registros, pero ocupa espacio y hace más costosas algunas escrituras. El proyecto ya tiene índices que podrían servir. Para elegir otro hay que revisar el plan con `EXPLAIN` y medir en datos representativos.

3. Dónde mirar en el código

Archivo	Responsabilidad
`app/Core/Paginacion.php`	Valida el número de página y calcula total, desplazamiento y páginas.
`app/Models/Oferta.php`	Listados de ofertas, candidatos paginados y conteo agrupado de plazas.
`app/Models/Asignacion.php`	Listado paginado, conteo de activas y próximos eventos del alumno.
`app/Models/Calificacion.php`	Calcula la media de calificaciones directamente en SQL.
`app/Models/Disponibilidad.php`	Comprueba si el alumno tiene disponibilidad sin traer toda la lista.
`app/Controllers/OfertaController.php`	Conecta las páginas de ofertas públicas, de empresa y de revisión.
`app/Controllers/AsignacionController.php`	Coordina páginas de asignaciones y matching.
`app/Controllers/InicioController.php`	Prepara el panel del alumno con consultas resumidas.
`app/Controllers/AlumnoController.php`	Entrega al portal del expediente una página de prácticas y el total.
`app/Views/partials/paginacion.php`	Genera los controles “Anterior” y “Siguiente”.
`app/Views/`	Las vistas de ofertas, matching, asignaciones y expediente muestran esos controles.
`scripts/benchmark_listados.php`	Mide consultas y tiempos en una base separada de benchmark.
`docs/optimizacion_consultas.md`	Documenta la configuración y el método de medición.

> **La vista no decide la compatibilidad.** La vista presenta candidatos; los modelos consultan y validan datos, y la asignación vuelve a comprobarlos al guardar.

4. Cómo probarlo y qué sabemos

Comprobaciones ya realizadas

- Los archivos PHP modificados pasan el comprobador de sintaxis.
- Se probaron límites de página y conservación de parámetros en los enlaces.
- `git diff --check` no detectó errores de formato.

Qué quedó pendiente

La prueba de integración necesita MariaDB. En la última ejecución el servidor local rechazó la conexión; por ello no se verificaron aún las consultas contra la base, los resultados de matching en ejecución ni los flujos completos con datos.

Medir con seguridad

1. Inicia MariaDB y prepara una base desechable llamada, por ejemplo, `fct_bench_octubre`.
2. Aplica el esquema y migraciones actuales; carga datos representativos y, si contienen información real, anonimizados.
3. Configura las variables `FCT_DB_HOST`, `FCT_DB_USER`, `FCT_DB_PASS` y `FCT_DB_NAME` para apuntar a esa base.
4. Ejecuta en PowerShell:

```
$env:FCT_BENCH_ALLOW = '1'
$env:FCT_BENCH_REPETITIONS = '7'
& 'C:\xampp\php\php.exe' scripts\benchmark_listados.php
```

El script se niega a correr si no se habilita explícitamente o si el nombre de la base no comienza por `fct_bench_`. También comprueba que haya un volumen mínimo de datos. No lo ejecutes en producción.

Compara medianas, número de consultas y filas mostradas usando la misma máquina, versión de MariaDB y copia de datos. Después revisa con `EXPLAIN` las consultas lentas. Solo entonces decide si algún índice adicional está justificado.

> **Resumen honesto:** los cambios están implementados y pasan validaciones estáticas. La comprobación real con MariaDB y la medición de rendimiento están pendientes de iniciar la base de datos y preparar una muestra adecuada.

Guía de apoyo para aprender los cambios del proyecto. Para los detalles de ejecución, consulta también `docs/optimizacion_consultas.md`.